

ZYVOR · NETRA

Network observability and emergency control, **without CNI lock-in.**

A buyer brochure for enterprises evaluating Netra — how every major capability works, how incidents move from signal to containment, and how Netra sits next to PacketWolf in the Zyvor suite.

OBSERVE · DIAGNOSE · CONTAIN · FAIL OPEN

Standalone eBPF for Linux/Kubernetes · Apache-2.0 · Optional Cilium/Hubble enrichment

Illustrative UI captures from a live lab deployment appear later in this brochure.

When enterprises buy Netra

Modern clusters accumulate network tools faster than operators can trust them. CNI dashboards explain policy verdicts. Host stacks still drop packets in softnet, qdisc, and the NIC. Emergency denies that never expire become the next outage. Netra is built for that gap.

Buyer problems Netra addresses

1. CNI-only visibility leaves node-level loss unexplained.
2. Changing or mixing CNIs should not erase observability and emergency control.
3. Incident kill-switches must auto-revert — permanent silent deny is unacceptable.
4. Evidence (capture, alerts, SIEM, AI) must not collect payloads, argv, or secrets.

What Netra is — in one sentence

Netra is standalone eBPF network observability and leased emergency containment for Linux/Kubernetes: observe-first, CNI-independent by default, with optional Cilium/Hubble enrichment.

Architecture — how it works

Netra has two workloads: a controller (netrad) that serves the API/UI and holds desired state, and a privileged node agent that owns eBPF programs and maps under `/sys/fs/bpf/netra`.



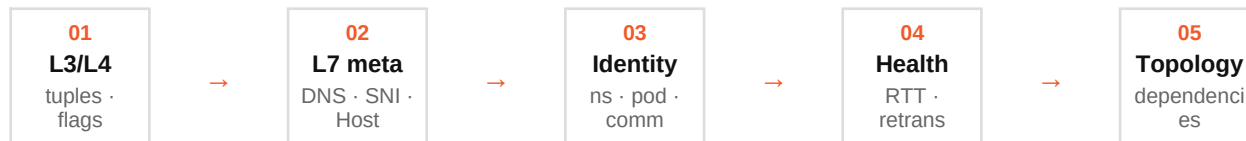
Hook model (default)

- `cgroup_skb ingress/egress` — CNI-independent workload observation and control
- `cgroup connect/sendmsg` — process/UID context and deny for new sockets
- `sockops` — TCP health, RTT, retransmit, connect latency
- Optional: `kfree_skb` drop reasons, TCX, XDP Shield on selected interfaces

The agent never pins over Cilium-owned maps. Cilium and Hubble are read as optional data sources.

Observability

Netra records flow counters, direction, hook attribution, TCP/UDP/ICMP visibility, Kubernetes workload identity, and metadata-only L7 signals.



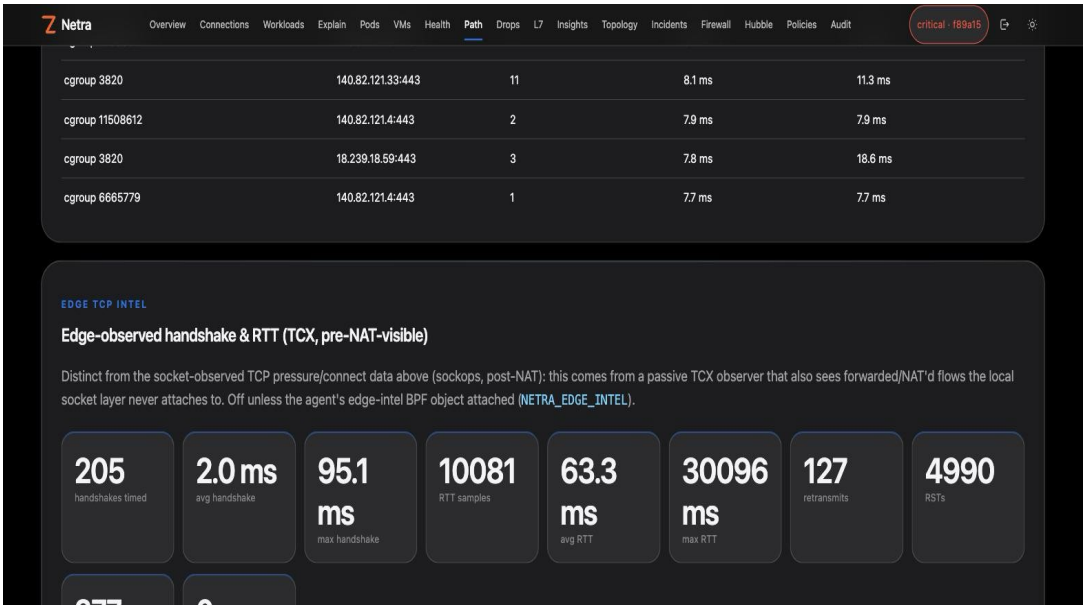
Hard boundary

No application payloads. No argv/cmdline. No Secret contents. DNS is ordinary UDP/53 query names. TLS is best-effort ClientHello SNI in a single skb. HTTP is method + Host only.

Path diagnostics & edge TCP intel

Path Diagnostics measures active TCP connect-establishment latency and exports transport pressure (cwnd, packets_out, retransmits, lost_out, delivered rate) per cgroup/workload.

A standalone TCX edge observer adds handshake/RTT histograms and retransmit/RST/FIN counters that see pre-NAT forwarded flows the local socket layer never attaches to.

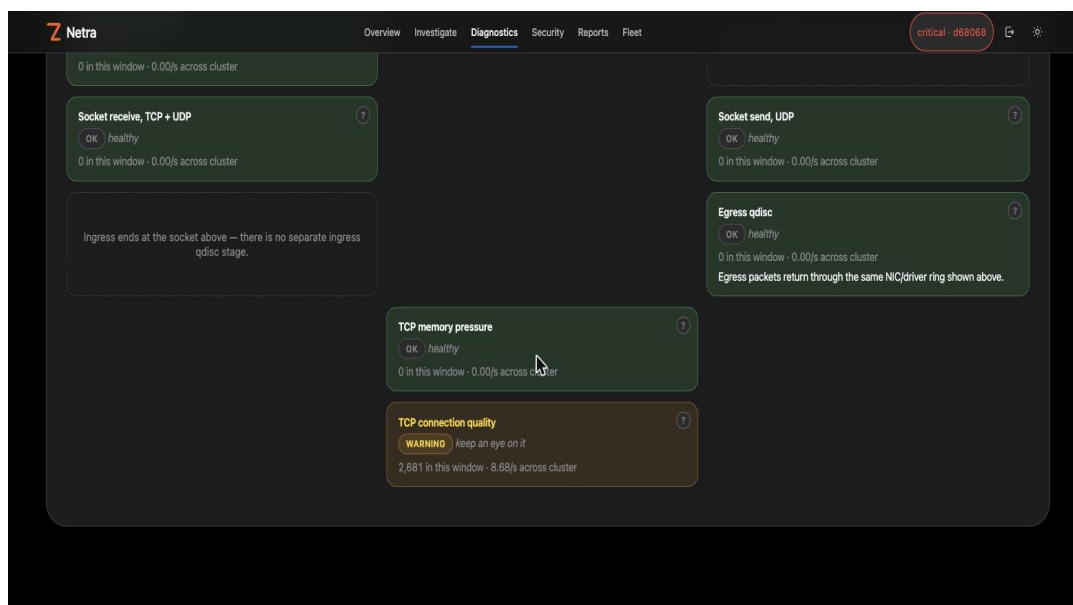


Illustrative lab UI — Path Diagnostics with edge TCP intel.

Drop diagnostics & Congestion Map

Drop Diagnostics combines optional `kfree_skb` drop-reason counters with Linux softnet, interface, and qdisc statistics — answering where packets die when CNI verdicts look fine.

The Congestion Map paints every stack stage across the cluster by severity. Critical findings deep-link to Capture; opt-in auto-capture can persist a PCAP automatically.



Illustrative lab UI — Congestion Map colored by severity.

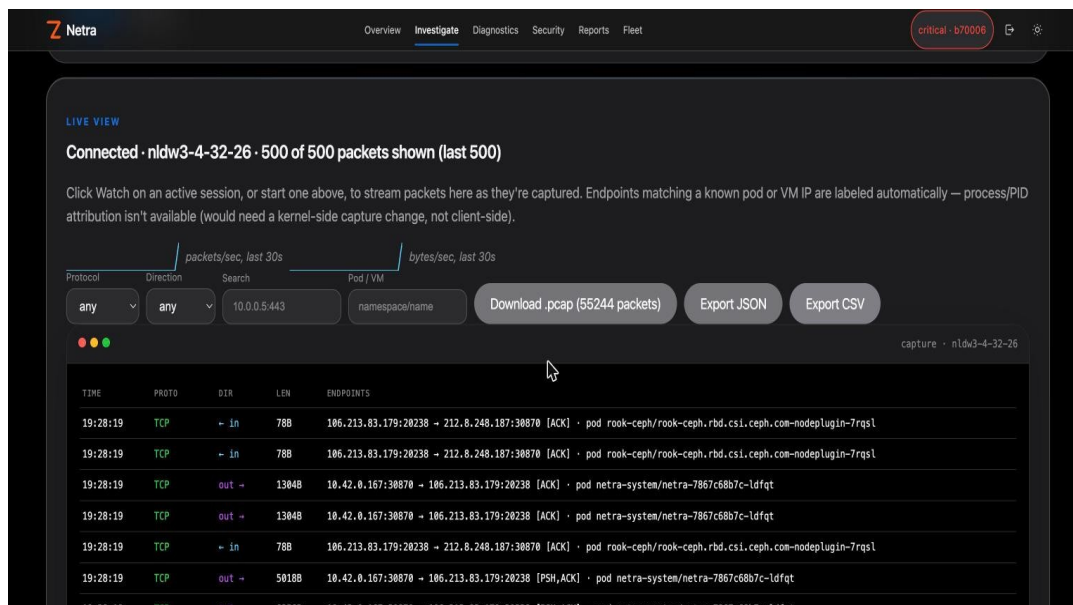
Netra never writes a sysctl. Tuning guidance is review-only and reversible.

Packet capture & auto-capture

Operators start a filtered, time-bounded capture on one or many nodes. Backends: eBPF (default, in-kernel) or AF_PACKET (userspace). Live stream, Wireshark-style decode, and .pcap export.

Auto-capture (opt-in)

When `NETRA_AUTO_CAPTURE` is enabled, critical softnet drops (≥ 1000), Congestion Map critical findings, and critical drop-rate spikes start a filtered capture and write a classic PCAP under `NETRA_AUTO_CAPTURE_DIR` — linked from capture history for download.



Illustrative lab UI — Capture live view.

Emergency firewall — leased enforce

Netra's containment layer is deliberately an emergency/containment tool — not a full CNI policy engine, QoS system, L7 proxy, or IDS/IPS.

What you can deny (examples)

- Exact IPv4/IPv6 and CIDR deny (ingress/egress), optional SYN-drop mode
- Port, DNS name, TLS SNI, UID, process comm — with allow-exceptions
- PPS/BPS ceilings and per-workload new-TCP connection-rate caps
- Optional XDP early ingress on selected interfaces

Lease lifecycle



Blast-radius preview matches a proposed deny against live counters and applies nothing — so operators can see impact before they confirm.

Behavior & rate insights

Controller-side insights turn eBPF metadata into baselines and drift: workload dependency graph, known-good behavior baseline, newly observed destinations/DNS/SNI/HTTP hosts/ports, time-window rate baselines with deterministic multipliers, exposure scoring, and review-only remediation proposals.

Netra never auto-applies learned policy. Baselines are explicit operator decisions.

Alerting & export

Multi-channel notify (webhook, email, Slack, Teams, Twilio SMS/WhatsApp, HTTP bridge) runs on an interval with severity-aware cooldown. HA: leader-only. Pull SIEM export supports JSON, JSONL, CEF, syslog, and OTLP. Prometheus gauges and an importable Grafana dashboard ship with the product.

AI, MCP & ChatOps

Ask Netra, digests, suggestions, explain, and draft are read-only — they never flip enforce mode or apply policy. Optional OpenAI-compatible rewrite; heuristic by default. netra-mcp exposes 133 tools (mutations gated off by default). Slack/Teams ChatOps mirrors the same confirmations as the UI.

Safety, HA, and deploy

Safety

Observe-first. Plan-token + risk confirm. Fail open on lease expiry, agent loss, restart, or HA handoff. No Cilium map overwrite. No payload/argv/Secret collection.

HA

Active/passive Lease election + shared RWX state. Auto-capture PCAPs should sit on the same shared volume as state so failover does not orphan artifacts.

Deploy via Helm (controller + optional agent DaemonSet). netra-doctor verifies host readiness (cgroup v2, bpffs, BTF) before privileged agent rollout.

Zyvor suite placement — Netra & PacketWolf

Netra and PacketWolf are suite counterparts, not a wired pipeline. Neither product is a dependency or API consumer of the other today.

	Netra	PacketWolf
Default market	Mixed / non-Cilium; OSS entry	Cilium-standardized fleets
Core job	Observe + Path/Drop/Congestion + leased emergency deny	Observe + AutoPolicy + healer + platform containment
CNI dependency	None (Cilium/Hubble optional)	Cilium-first
Policy posture	Review-only drafts; enforce is time-leased, fails open	Learns and operates Cilium policy at platform depth
Maps	Owns /sys/fs/bpf/netra only	Works with Cilium datapath / policy
Relationship	Suite counterpart — not a PacketWolf agent or API consumer	Suite counterpart — no Netra control-plane sync today

Co-existence rules

5. Same cluster is allowed but not required — Netra maps stay isolated.
6. Do not dual-own long-lived deny: PacketWolf/Cilium for durable posture; Netra leases for incident kill-switch that auto-reverts.
7. Enable Hubble/CNP in Netra only for enrichment — not as PacketWolf's datapath.
8. There is no Netra ↔ PacketWolf control-plane sync today.

Pick Netra when you need CNI-independent diagnostics and short-lease emergency deny. Pick PacketWolf when Cilium is already the CNI and you want full platform intelligence / AutoPolicy / healer depth.

Buying checklist

1. Confirm Linux nodes meet cgroup v2 / bpffs / BPF baselines (netra-doctor).
2. Start observe-only: flows, Path, Drop, Congestion Map — no enforce lease yet.
3. Exercise capture (and optionally auto-capture) on a non-production signal.
4. Pilot a leased deny with blast-radius preview and confirm auto-revert.
5. Wire alerting / SIEM / ChatOps as needed; keep AI/MCP read-only until trusted.
6. If Cilium is present, decide enrichment vs PacketWolf platform ownership explicitly.

See the network. Contain the incident. Fail open by default.

Talk to sales → zyvor.dev/netra

© 2026 Zyvor · Apache-2.0 core · Standalone eBPF network observability and emergency network control